

并行处理与体系结构(8) Parallel Processing and Architecture

2025-05-25

1

回顾：并行性

▶ Amdahl定律

- ▶ f: 程序中可并行的部分
- ▶ N: 处理器个数

$$\text{加速比} = \frac{1}{1-f + \frac{f}{N}}$$

- ▶ Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

▶ 最大加速比受限于串行部分: 串行瓶颈

▶ 并行部分通常不完美

- ▶ 同步开销 (比如, 对共享数据的更新)
- ▶ 负载不均衡开销 (并行化不完美)
- ▶ 资源共享开销 (N个处理器之间的竞争)

▶

2

回顾：利用不对称

- ▶ 串行部分的执行时间必须短
- ▶ 对程序员来说缩短这些串行段相当困难
 - ▶ 领域知识不够
 - ▶ 硬件平台的多样性
 - ▶ 受限的资源
- ▶ 目标: 一种不需要程序员参与的缩小串行瓶颈的机制
- ▶ 思路: 在非对称多核平台上通过将串行代码段迁移到强有力的核上来加速串行部分的执行

▶

3

3

回顾：加速并行瓶颈

- ▶ 并行部分中的序列化或不均衡的执行同样可以得益于大核
- ▶ 例子:
 - ▶ 临界区竞争
 - ▶ 比别的阶段执行时间更长的并行阶段
- ▶ 思路: 动态判别会导致序列化执行的代码段并将它们放到大核上执行
 - ▶ 加速临界区
 - ▶ 瓶颈识别和调度

▶

4

4

回顾：多线程应用中的瓶颈

一种定义：任何会被线程竞争的代码段

▶ Amdahl的串行段

- ▶ 只有一个线程在执行 → 在关键路径上

▶ 临界区

- ▶ 保证互斥 → 如果有竞争很可能就在关键路径上

▶ 栅障

- ▶ 再继续推进之前保证所有线程到达该点 → 最后到达的线程在关键路径上

▶ 流水线阶段

- ▶ 一个循环迭代的阶段可能在不同线程上执行，最慢的阶段会使其它阶段等待 → 在关键路径上

5

5

回顾：有什么收获？

- ▶ 硬件并行在很多方面都得到发展

- ▶ 能够获得更大的性能提升？

6

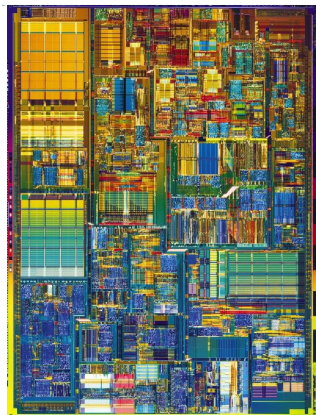
6

回顾：处理器性能瓶颈？

晶体管 →

功率 →

软件 →



7

7

回顾：启示？

- ▶ 关注功率，不是晶体管数

- ▶ 1. 通过优化“性能/瓦”来优化性能(不是“性能/晶体管数”)

- ▶ 2. Dark Silicon

- ▶ 利用率墙—某些晶体管必须总是关闭

- ▶ “怎样才能通过增加会被关掉的晶体管来增加处理器的价值？”

- → heterogeneity

8

8

回顾：启示？

▶ 关注软件，不是晶体管数

- ▶ 1. 许多核，不是许多线程
- ▶ 2. 用任何可能的方式增加并行性——体系结构、编译器、编程语言、编程工具.....
- ▶ 3. 即使我们没有并行性，仍然需要并行加速
- ▶ 4. 线程(运行这些线程的核或SMT上下文)是“免费的”



9

回顾：软件并行性的来源

- ▶ 多道程序
- ▶ 并行编程
- ▶ 并行化编译器
- ▶ 并行语言
- ▶ 串行代码动态并行机制（例如，通过动态编译）
- ▶ 投机多线程
- ▶ helper线程
- ▶



10

回顾：并行软件

- ▶ 由软件负责并行
 - ▶ 硬件和编译器可以跟上所需的速度
- ▶ 软件并行...
 - ▶ 在共享内存程序中
 - ▶ 启动单个进程并创建线程
 - ▶ 线程执行任务
 - ▶ 在分布式存储程序中
 - ▶ 启动多个进程
 - ▶ 进程执行任务



11

回顾：写并行程序

- ▶ 在进程/线程之间分配工作
 - ▶ 每个进程/线程得到的工作量大致相同
 - ▶ 并且通信被最小化
- ▶ 安排进程/线程同步
- ▶ 安排进程/线程之间的通信



12

回顾：并行编程的Foster方法论

- 1. 划分：把要执行的计算和计算操作的数据划分成小任务
 - 这里的重点应该是确定可以并行执行的任务
- 2. 通信：确定在上一步确定的任务中需要进行什么样的通信
- 3. 聚集或聚合：将第一步中确定的任务和通信组合成更大的任务
 - 例如，如果任务A必须在任务B执行之前执行，那么将它们聚合成一个复合任务可能是有意义的
- 4. 映射：将上一步中确定的复合任务分配给进程/线程
 - 这样做可以使通信最小化，并且每个进程/线程获得大致相同的工作量

▶

13

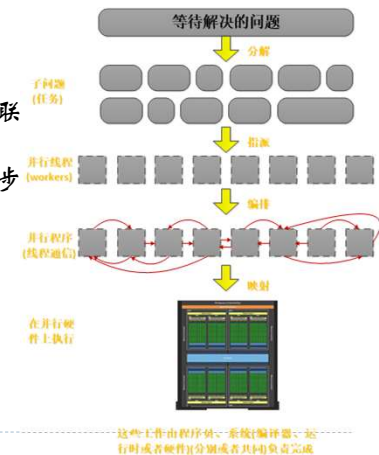
回顾：创建并行程序

▶ 思考：

1. 明确可以并行的任务
2. 划分任务（包括与任务关联的数据）
3. 处理数据读写、通信和同步

- ▶ 并行的主要目标是加速比（speedup）

$$\text{speedup}_p = T_1 / T_p$$



▶

14

回顾：分解

- ▶ 将问题分解成可以并行执行的任务
 - 分解是动态的
 - 随着程序的执行，需要识别新的任务

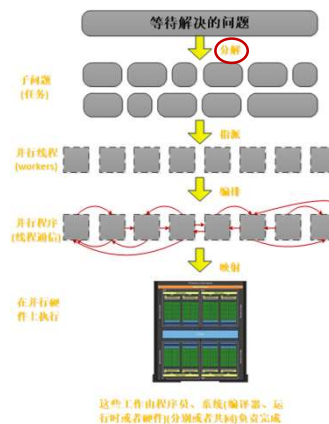
- ▶ 主要思路：想办法创建足够多的任务使得系统中所有的执行单元保持忙碌

分解的关键：识别依赖性

阿姆达尔定律 (Amdahl's Law) ——
依赖性限制了最大的并行加速比

设S为串行程序中由于依赖性的限制而无法并行的部分
则并行执行可以获得的最大加速比 $\leq 1/S$

▶



15

回顾：一个简单的例子

- ▶ 设想一个分两步计算处理的 $N \times N$ 图像
 - ▶ 第一步：将所有像素点的亮度加倍（每个像素的计算互相独立）
 - ▶ 第二步：计算所有像素值的平均值

▶ 串行执行

▶ 每一步执行时间为 N^2 ，总执行时间为 $2N^2$

▶ 初步尝试并行

▶ 策略

▶ 第一步：并行执行，执行时间为 N^2/p

▶ 第二步：串行执行，执行时间为 N^2

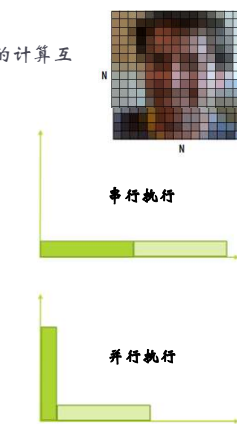
▶ 总执行时间为 $N^2/p + N^2$

▶ 总体性能

$$\text{Speedup} \leq 2N^2 / (N^2/p + N^2)$$

$$\text{Speedup} \leq 2$$

▶



16

回顾：进一步并行

策略

- ▶ 第一步：并行执行，执行时间为 N^2/p
- ▶ 第二步：并行计算部分和，串行计算总和，执行时间为 $N^2/p+p$

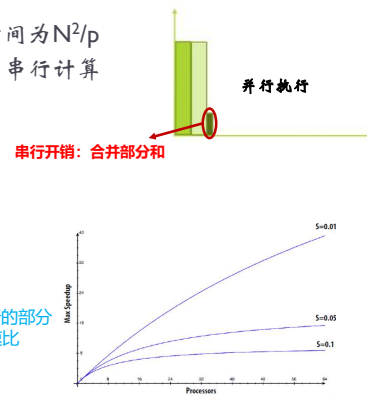
总体性能

$$\text{Speedup} \leq 2N^2/(2N^2/p+p)$$

当 $N \gg p$ 时， $\text{Speedup} \rightarrow p$

阿姆达尔定律 (Amdahl's Law)

设 S 为串行程序中由于依赖性的限制而无法并行的部分
则在 p 个处理器上并行执行可以获得的最大加速比
 $\text{Speedup} \leq 1/(S+(1-S)/p)$



17

回顾：分解

谁来负责任务分解？

- ▶ 通常来说，由程序员负责

串行程序的自动任务分解始终是一个具有挑战性的研究问题（通常相当难）

- ▶ 编译器必须分析程序，识别依赖性
 - ▶ 有些数据依赖在编译时无法知晓
- ▶ 研究人员在简单的循环嵌套上取得了一定的成功
- ▶ 能够处理复杂、通用代码的“魔法并行编译器”还有很多工作要做

18

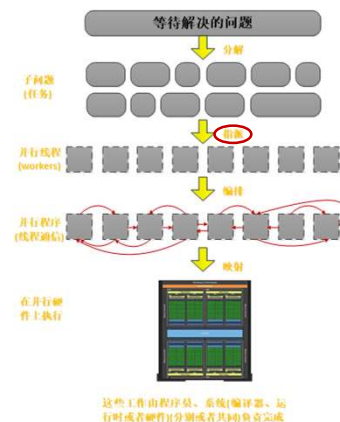
指派

将任务分配给线程

- ▶ 任务 → 要完成的工作
- ▶ 线程 → 完成工作的人

目标：负载均衡，降低通信开销

- ▶ 可以静态完成，也可以运行时动态完成
- ▶ 虽然任务分解通常由程序员负责，但是通常语言/运行时负责完成指派



19

Intel ISPC编译器指派的例子

```
export void sinc(
    uniform int N,
    uniform int terms,
    uniform float* x,
    uniform float* result)
{
    // assumes N % programCount == 0
    for (uniform int i=0; i<N; i+=programCount)
    {
        int idx = i + programIndex;
        float value = x[idx];
        float numer = x[idx] * x[idx] * x[idx];
        uniform int denom = 6; // 3!
        uniform int sign = -1;

        for (uniform int j=1; j<terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[idx] * x[idx];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[i] = value;
    }
}
```

循环迭代任务的分解

程序员负责的指派：静态指派

以交错的方式将迭代指派给ISPC程序实例

```
export void sinc(
    uniform int N,
    uniform int terms,
    uniform float* x,
    uniform float* result)
{
    foreach (i = 0 ... N)
    {
        float value = x[i];
        float numer = x[i] * x[i] * x[i];
        uniform int denom = 6; // 3!
        uniform int sign = -1;

        for (uniform int j=1; j<terms; j++)
        {
            value += sign * numer / denom;
            numer *= x[i] * x[i];
            denom *= (2*j+2) * (2*j+3);
            sign *= -1;
        }
        result[i] = value;
    }
}
```

foreach结构体向系统展示独立的任务

系统负责将任务(迭代)指派给ISPC程序实例

(语言层面的抽象为动态分配留下了空间，但是当前的ISPC实现是静态的)

20

使用Pthread进行静态指派的例子

```
typedef struct {
    int N, terms;
    float* x, *result;
} my_args;

void parallel_sin(int N, int terms, float* x, float* result)
{
    pthread_t thread_id;
    my_args args;

    args.N = N/2;
    args.terms = terms;
    args.x = x;
    args.result = result;

    // launch second thread, do work on first half of array
    pthread_create(&thread_id, NULL, my_thread_start, &args);

    // do work on second half of array in main thread
    sinx(N - args.N, terms, x + args.N, result + args.N);

    pthread_join(thread_id, NULL);
}

void my_thread_start(void* thread_arg)
{
    my_args* thread_args = (my_args*)thread_arg;
    sinx(thread_args->N, thread_args->terms, thread_args->x, thread_args->result); // do work
}
```

循环迭代任务的分解

程序员负责的指派：静态指派
程序以阻塞的方式将迭代分配给Pthreads

数组的前半部分分配给衍生线程，后半部分分配给主线程

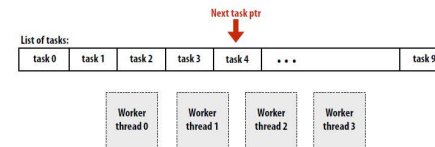


21

利用ISPC任务进行动态指派

```
void foo(uniform float* input,
        uniform float* output,
        uniform int N)
{
    // create a bunch of tasks
    launch[100] my_ispc_task(input, output, N);
}
```

ISPC运行时指派任务给worker线程



实现任务向线程的指派：当前任务完成后，worker线程检查任务列表并且分配下一个未完成的线程给自己

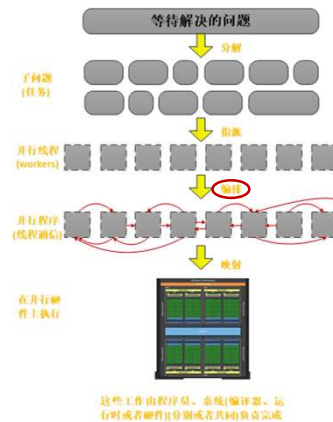


22

编排

包括：

- 建立通信
- 如有必要，添加同步保护依赖
- 组织内存中的数据结构
- 调度任务



23

映射到硬件

映射“线程”(worker)到硬件执行单元

例1：由操作系统映射

- 比如映射Pthread到CPU核上的硬件执行上下文

例2：由编译器映射

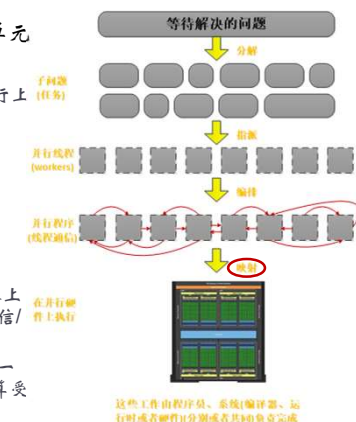
- 映射ISPC程序实例到向量指令通道

例3：由硬件映射

- 映射CUDA线程块到GPU核

一些有趣的映射策略

- 将相关线程(协作线程)放在同一处理器上(最大化局部性和数据共享，最小化通信/同步开销)
- 将不相关的线程放在同一个处理器上(一个可能是带宽受限，另一个可能是计算受限)更有效地使用机器



24

基于2维网格的求解器

- 在 $(N+2) \times (N+2)$ 网格上求解偏微分方程(PDE)

- 迭代求解

- 在网格上执行高斯-赛德尔法，直到收敛

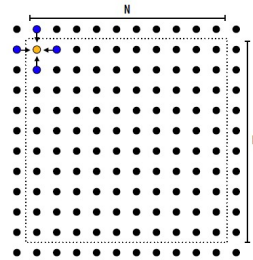
```
const int n;
float* A; // assume allocated to grid of N+2 x N+2 elements

void solve(float* A) {
    float diff, prev;
    bool done = false;

    while (!done) { // outermost loop: iterations
        diff = 0.f;
        for (int i=1; i<n; i++) { // iterate over non-border points of grid
            for (int j=1; j<n; j++) {
                prev = A[i,j];
                A[i,j] = 0.25 * (A[i,j-1] + A[i,j+1] + A[i-1,j] + A[i+1,j]);
                diff += fabs(A[i,j] - prev); // compute amount of change
            }
        }

        if (diff/(n*n) < TOLERANCE) // quit if converged
            done = true;
    }
}
```

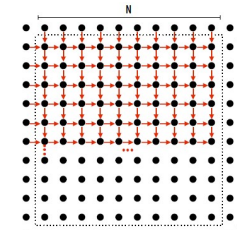
网格求解器算法：串行算法的类C伪代码



25

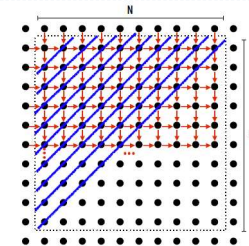
第一步：识别依赖性

- 问题分解阶段



行中的每一个元素依赖其左侧的元素，每一行依赖前一行

注：上图显示的依赖关系是求解器在一次迭代中的元素数据依赖关系(在“尚未完成”的循环的一次迭代中)



沿着对角线有独立的工作

优点：存在并行性

可能的实施策略：

1. 将对角线上的网格单元划分成任务
2. 并行更新值
3. 完成后，移动到下一个对角线

缺点：独立的工作很难被利用
计算开始和结束时没有太多并行性；
需要频繁同步(完成每条对角线后)

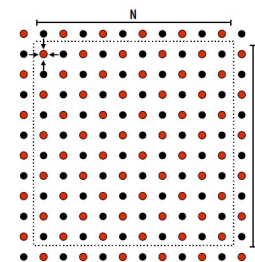
26

能不能更轻松一点儿？

- 思路：通过把算法改得更容易并行来改善性能
 - 更改网格单元的更新顺序
 - 新算法会迭代到大致相同的解，但会收敛到不同的解
 - 浮点数计算的结果不同，但是解仍然会收敛到误差阈值之内
- 确实需要高斯赛德尔方法求解线性系统的领域知识，才能实现应用能够接受的变化

27

新方法:通过红黑着色重新排序网格单元更新



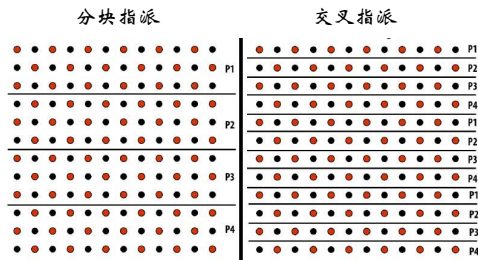
并行更新所有的红色单元

红色单元更新完成后，并行更新所有黑色单元(注意与红色单元的依赖性)

重复更新直至收敛

28

向处理器指派任务的可能方式



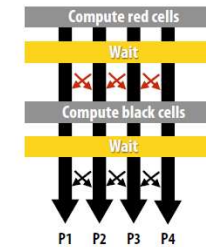
问题：哪一种方式更好？是否重要？

答案：这取决于程序所运行的系统

29

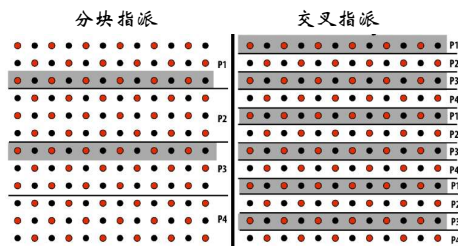
考虑依赖(数据流)

- 1、并行执行红色单元的更新
- 2、直到所有处理器完成更新
- 3、与其他处理器通信更新红色单元
- 4、并行执行黑色单元的更新
- 5、直到所有处理器完成更新
- 6、与其他处理器通信更新黑色单元
- 7、重复



30

指派导致的通信



灰色区域：每次迭代必须发送给相邻处理器的数据

分块指派需要在处理器间通信的数据相对更少

31

网格求解器的数据并行表达

简化伪代码：只看红色单元的更新

```
const int n;
float* A = allocate(n+2, n+2); // allocate grid 指派
void solve(float* A) {
    bool done = false;
    float diff = 0.0;
    while (!done) {
        for_all (red cells (i,j)) {
            float prev = A[i,j];
            A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j] +
                          A[i+1,j] + A[i,j+1]);
            reduceAdd(diff, abs(A[i,j] - prev));
        }
        if (diff/(n*n) < TOLERANCE)
            done = true;
    }
}
```

分解：独立的网格元素
构成独立的任务

编排：由系统处理（内嵌在通信
原语中，reduceAdd）

编排：由系统处理（for_all
结束意味着在返回顺序控制
之前等待所有worker）

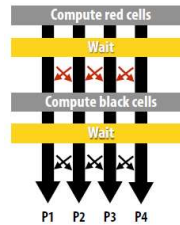
32

求解器的共享地址空间表达

▶ 程序员负责同步

▶ 常用同步原语

- ▶ 锁(提供互斥): 同一时刻只有一个线程在临界区
- ▶ barrier: 等待线程到达这一点



33

共享地址空间求解器(SPMD 执行模式)

假设这些是全局变量 (可以被所有线程访问)

```
int n; // grid size
bool done = false;
float diff = 0.0;
LOCK mylock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n+2, n+2);

void solve(float* A) {
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS);

    while (!done) {
        diff = 0.0;
        barrier(myBarrier, NUM_PROCESSORS);
        for (j = myMin to myMax) {
            for (i = red cells in this row) {
                float prev = A[i,j];
                A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j+1] + A[i+1,j]);
                lock(mylock);
                diff += abs(A[i,j] - prev);
                unlock(mylock);
            }
        }
        barrier(myBarrier, NUM_PROCESSORS);
        if (diff/(n*n) < TOLERANCE)
            done = true;
        barrier(myBarrier, NUM_PROCESSORS);
    }
}
```

假设求解函数被所有线程执行 (SPMD)

每个SPMD实例的线程ID号不同: 用ID号计算不同的网格区域

每个线程计算所负责更新区域的行

34

需要互斥

每一个线程执行

load diff的值写入寄存器r1

寄存器r2的值加r1的值写入r1

r1的值store进diff

一种可能的执行 (假设初始状态 diff=0, r2=1)

一组三条指令必须是原子的

T0	T1
r1 ← diff	r1 ← diff
r1 ← r1 + r2	r1 ← r1 + r2
diff ← r1	diff ← r1

35

共享地址空间求解器

```
int n; // grid size
bool done = false;
float diff = 0.0;
LOCK mylock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n+2, n+2);

void solve(float* A) {
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS);

    while (!done) {
        diff = 0.0;
        barrier(myBarrier, NUM_PROCESSORS);
        for (j = myMin to myMax) {
            for (i = red cells in this row) {
                float prev = A[i,j];
                A[i,j] = 0.2f * (A[i-1,j] + A[i,j-1] + A[i,j+1] + A[i+1,j]);
                lock(mylock);
                diff += abs(A[i,j] - prev);
                unlock(mylock);
            }
        }
        barrier(myBarrier, NUM_PROCESSORS);
        if (diff/(n*n) < TOLERANCE)
            done = true;
        barrier(myBarrier, NUM_PROCESSORS);
    }
}
```

这个实现中有一个潜在的性能问题, 看出来了吗?

36

共享地址空间求解器

```
int n; // grid size
bool done = false;
float diff = 0.0;
LOCK mylock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n*2, n*2);

void solve(float* A) {
    float myDiff;
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS);

    while (!done) {
        float myDiff = 0.0;
        diff = 0.0;
        barrier(myBarrier, NUM_PROCESSORS);
        for (j=myMin to myMax) {
            for (i = red cells in this row) {
                float prev = A[i,j];
                A[i,j] = 0.25 * (A[i-1,j] + A[i,j-1] + A[i,j] +
                    A[i+1,j], A[i,j+1]);
                myDiff += abs(A[i,j] - prev);
            }
        }
        lock(mylock);
        diff += myDiff;
        unlock(mylock);
        barrier(myBarrier, NUM_PROCESSORS);
        if (diff/(n*n) < TOLERANCE) // check convergence, all threads get same answer
            done = true;
        barrier(myBarrier, NUM_PROCESSORS);
    }
}
```

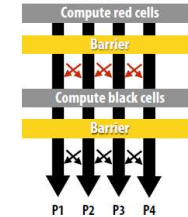
通过在局部累加成分和，
然后在迭代结束时在全局
完成归约来提高性能

每个worker计算部分和
现在每个线程一个锁，而不是
每个ij循环迭代一个锁

37

Barrier同步原语

- ▶ barrier(num_threads)
- ▶ barrier是一种保守的表示依赖的方式
- ▶ barrier将计算分成阶段
- ▶ 在barrier之前的所有线程的所有计算完成之前，barrier之后的所有线程的所有计算不能开始
- ▶ 换句话说，barrier之后的所有计算可以被认为是依赖于barrier之前的所有计算



38

共享地址空间求解器

```
int n; // grid size
bool done = false;
float diff = 0.0;
LOCK mylock;
BARRIER myBarrier;

// allocate grid
float* A = allocate(n*2, n*2);

void solve(float* A) {
    float myDiff;
    int threadId = getThreadId();
    int myMin = 1 + (threadId * n / NUM_PROCESSORS);
    int myMax = myMin + (n / NUM_PROCESSORS);

    while (!done) {
        float myDiff = 0.0;
        diff = 0.0;
        barrier(myBarrier, NUM_PROCESSORS);
        for (j=myMin to myMax) {
            for (i = red cells in this row) {
                float prev = A[i,j];
                A[i,j] = 0.25 * (A[i-1,j] + A[i,j-1] + A[i,j] +
                    A[i+1,j], A[i,j+1]);
                myDiff += abs(A[i,j] - prev);
            }
        }
        lock(mylock);
        diff += myDiff;
        unlock(mylock);
        barrier(myBarrier, NUM_PROCESSORS);
        if (diff/(n*n) < TOLERANCE) // check convergence, all threads get same answer
            done = true;
        barrier(myBarrier, NUM_PROCESSORS);
    }
}
```

为什么有三个barrier?

39

共享地址空间求解器: 一个barrier

```
int n; // grid size
bool done = false;
LOCK mylock;
BARRIER myBarrier;
float diff[3]; // global diff, but now 3 copies

float* A = allocate(n*2, n*2);
```

思路:

在连续的循环迭代中使用
不同的 diff 变量消除依赖

```
void solve(float* A) {
    float myDiff; // thread local variable
    int index = 0; // thread local variable

    diff[0] = 0.0f;
    barrier(myBarrier, NUM_PROCESSORS); // one-time only: just for init

    while (!done) {
        myDiff = 0.0f;
        // perform computation (accumulate locally into myDiff)
        lock(mylock);
        diff[index] += myDiff; // atomically update global diff
        unlock(mylock);
        diff[(index+1) % 3] = 0.0f;
        barrier(myBarrier, NUM_PROCESSORS);
        if ((diff[index]/(n*n)) < TOLERANCE)
            break;
        index = (index + 1) % 3;
    }
}
```

通过牺牲空间消除依赖
(常用的并行编程技巧)

40

期末大作业

▶ 基础题（60分）

- ▶ 1. 题目描述
- ▶ 2. 提交说明

▶ 进阶题（40分）

- ▶ 1. 题目描述
- ▶ 2. 提交说明
- ▶ 3. 附加说明

▶ 评分细则

▶ 服务器使用说明

- ▶ 基本信息
- ▶ 使用说明

▶ 41

41

基础题（60分）

- ▶ 1、使用C/C++语言设计编写一个包含不少于三种并行性能瓶颈的并行程序，在源代码中用注释标明瓶颈位置。对存在的问题进行相应的优化，给出优化后的源代码。对优化策略和加速效果在文档中进行详细分析，最好配有清晰明确的图表辅助说明

- ▶ 最后提交的代码应尽可能完成一个具体且有意义的功能，其中存在的并行性能瓶颈应包含但不仅限于缓存同步、线程同步等

▶ 2、提交说明

- ▶ 基础题应提交两份C/C++源代码（其中包括一份包含不少于3个并行瓶颈的源文件和一份解决了所有瓶颈的源文件）和一份说明文档。两份源代码也需要上传至服务器，并保证在服务器上可以正确编译运行

▶ 42

42

基础题（60分）——一个例子

```
1 // 2 该程序的功能是对于一个规模为M的数组中的每个元素进行统计  
3 每个元素的出现次数 (M = 1)  
4 遍历所有 (M = 1) 中每个元素在数组中出现的次数记录在一个数组中并输出  
5 程序采用多线程并行的方式，线程数为T  
6 遍历所有元素并统计其出现次数并输出到数组中的函数数据  
7 //  
8 #include <iostream>  
9 #include <random>  
10 #include <pthread.h>  
11 using namespace std;  
12  
13 #define N 10000  
14 #define M 10  
15 #define T 2  
16  
17 int input[N];  
18 int result[T];  
19 int thread_data[T]; // 存在于一个缓存行当中  
20 int index_bulk = N / T;  
21  
22 pthread_mutex_t mutex_lock;  
23  
24 typedef struct {  
25     int start;  
26     int end;  
27     int index;  
28 }  
29 bool initialization() {  
30     srand48(generator);  
31     uniform_int_distribution<int> distribution(0, N);  
32     for (int i = 0; i < N; i++) {  
33         input[i] = distribution(generator);  
34     }  
35 }  
36  
37 void* worker(void* arg) {  
38     index_t index = *(index_t*)arg;  
39     int start = index.start;  
40     int end = index.end;  
41     int thread_id = (int)(start/index_bulk);  
42     for (int i = start; i <= end; i++) {  
43         thread_data[thread_id]++; // 这里存在缓存同步和共享问题  
44     }  
45     // 以下临界区的锁过小，导致加速瓶颈的产生  
46     pthread_mutex_lock(&mutex_lock);  
47     result[thread_id] += thread_data[thread_id];  
48     pthread_mutex_unlock(&mutex_lock);  
49 }  
50  
51 int main() {  
52     initialization();  
53     pthread_mutex_init(&mutex_lock, NULL);  
54     index_t arg_array[T];  
55     pthread_t thread_id[T];  
56     for (int i = 0; i < index_bulk; i++) {  
57         arg_array[i].start = i * index_bulk;  
58         arg_array[i].end = i * index_bulk + index_bulk - 1;  
59         pthread_create(&thread_id[i], NULL, worker, &arg_array[i]);  
60     }  
61     pthread_mutex_destroy(&mutex_lock);  
62 }
```

▶ 43

43

进阶题（40分）

▶ 1、题目描述

- ▶ 在课程给定的服务器上，对完整的应用程序进行分析和优化，主要的两大赛道为：

- ▶ (1) 缓存层次结构瓶颈；

- ▶ (2) 线程同步瓶颈

- ▶ 其中赛道1为必选赛道，赛道2为选做赛道

- ▶ 两大赛道分别由课程组提供一个benchmark程序包，大家可以使用任意工具对程序包进行分析，最终目标为达到最高的优化加速比。最终每个小组应在每个赛道中选取2个程序进行优化（程序的选取是任意的，但应在报告中说明选取的原因），每一个找出的问题和对应的优化方案应在文档中进行详细说明

▶ 44

44

进阶题（40分）

2、提交说明

- ▶ 进阶题应分赛道提交文件。每一个赛道应提交一份报告，报告内容格式不限，但内容应至少包括以下四部分：
 - ▶ 1.检测工具的选择和使用方法介绍
 - ▶ 2.工具检测结果报告分析（指出工具发现了哪些问题，每一个问题是否是真实有效的，从理论上分析为何会产生这些问题等）
 - ▶ 3.优化策略分析（说明从benchmark中选取了哪几个程序进行优化及其原因，说明从报告中选取哪些瓶颈进行优化及其原因，优化的策略和方法等）
 - ▶ 4.实验分析（使用图表分析说明应用程序整体的优化加速比、扩展性、以及对同一个瓶颈采用不同优化策略的优化效果对比分析等）
- ▶ 源代码无需提交，保留在服务器各小组对应的目录下即可。保证修改后的源代码能够正常编译运行，并且程序运行输出结果正确

▶ 45

45

进阶题（40分）

3、附加说明

- ▶ 对于检测工具，我们鼓励大家多多搜索、尝试使用不同的性能分析工具。在此，课程组给出一个推荐工具，方便大家更容易入手性能分析这一领域。由于该工具为课程组自行开发，可能会存在一些bug，因此如果在使用推荐工具期间发现了任何工具的漏洞或问题，欢迎通过gitee网站提交issue，经验证有效的bug会给提交者酌情加分。同时，若使用自行寻找的非推荐工具检测并解决了显著的性能瓶颈，也可酌情加分
- ▶ 工具仓库地址：DrCCTProf_cache_inefficiency_detection
- ▶ 工具参考文献：冯文韬,栾钟治,杨海龙,等.面向天河新一代超算系统通用处理器的性能分析工具集[J].计算机学报,2024,47(02):423-440.

▶ 46

46

评分细则

- ▶ (1) 基础题根据源代码编写质量、性能瓶颈的代表性及报告质量进行评分
- ▶ (2) 进阶题首先会对每个应用程序所对应的所有小组提交的优化版程序进行测速并排名，以所有提交的程序中最慢的（不慢于原程序）为基准，最快的为上限将程序运行速度线性映射到 $W \in [0.8, 1.5]$ 这一权重区间。同一小组在同一赛道下提交的两个应用程序的优化版本，以平均成绩计算该赛道得分权重。（若一个程序得分0.8，一个程序得分1.4，则该赛道得分权重为 $(0.8+1.4) \div 2 = 1.1$ ）
- ▶ 基础优化得分为每个赛道10分，最终单赛道优化得分 $S_i = 10 \times W_i$ ，报告文档总计20分
- ▶ 因此，进阶题总计得分 $S = \min(S_{\text{赛道1}} + S_{\text{赛道2}} + S_{\text{文档}}, 40)$

▶ 47

47

服务器使用说明

基本信息

- ▶ 课程组为大家准备好了一台ARM架构服务器，128核Phytium S2500，操作系统为KylinV10，其具体信息如下：（！只能在校网下连接！）

服务器信息		Shell
1	服务器ip: 10.254.50.68	
2	用户名: para_arch_course	
3	密码: Nova_2024	

- ▶ 在该用户文件夹结构如下：

- ▶ track_1和track_2文件夹下均包含一个benchmark文件夹，每个benchmark的编译、运行和源代码修改方式可参考gitee仓库介绍。splash2, parsec3.0 tools文件夹内部包含推荐工具源码，其使用方式参考gitee仓库

```
tools
├── DrCCTProf-cache_inefficiency_manner_check
├── track_1
├── splash2_benchmark-master
├── track_2
└── parsec_for_optimization
```

▶ 48

48

服务器使用说明

▶ 使用说明

- ▶ 每一组在用户根目录下新建一个文件夹，命名为group_x。
大家将所需的内容从上述三个文件夹中拷贝到自己的私有目录下进行使用和修改，禁止直接在tools, track_1, track_2三个目录下进行任何操作!!!

- ▶ 最终，每一组在服务器上提交留存的最后文件应为如下结构：

```
[root@localhost group_1]# tree -L 2
├── basic
│   ├── bottleneck_optimization.c
│   └── bottleneck_source.c
└── extra
    ├── track_1
    └── track_2
```

- ▶ 其中extra文件夹下面的track1和track2文件夹内部应各保存2个所选程序文件夹，内部包含优化后的程序源代码和可执行文件

▶ 49

课程成绩

▶ 分组报告 60%

- ▶ 各组提交汇报ppt
- ▶ 时间节点：6月29日截止

▶ 期末大作业 40%

- ▶ 分组完成大作业
 - ▶ 按调研时的分组，一共7组
 - ▶ 助教在群中，大家有任何问题可在群中提出讨论
 - ▶ 大作业中请注明每位组成员的具体工作内容
- ▶ 时间节点
 - ▶ 6月29日截止

▶ 50